# Modeling Realistic SCADA Traffic: Fingerprinting and Diffusion

Milin Doppalapudi
*midoppal@ucsc.edu*

Aidan Gray
*aitgray@ucsc.edu*

Charlie Alders
*calders@ucsc.edu*

Ramon Torres
*rtorre25@ucsc.edu*

## Abstract

We present a framework for generating realistic Modbus-based SCADA traffic using diffusion models, traffic fingerprinting, and automated validation. Our approach analyzes real and synthetic SCADA datasets to characterize protocol behavior and timing patterns. We evaluate NetDiffusion for SCADA traffic generation and identify practical limitations when payload semantics and request-response concurrency is required. To address these challenges, we develop a custom diffusion pipeline trained on structured Modbus transaction features, enabling the generation of protocol-compliant synthetic traffic.

We validate generated traffic using a multi-stage pipeline that combines protocol-level verification with statistical evaluation. Protocol correctness is assessed through transaction matching and virtual server interaction, while statistical similarity is measured using principle component analysis and linear classification. Experimental results show that the generated traffic preserves key structures and statistical properties of real Modbus communication, with a non-negligible fraction of synthetic packets overlapping real traffic in feature space. These results demonstrate the feasibility of diffusion-based SCADA traffic synthesis, highlighting directions for future refinement.

## 1 Introduction

Industrial Control System (ICS) research is fundamentally constrained by limited access to realistic, high-quality network datasets. Unlike traditional enterprise networks, ICS environments are operated by critical infrastructure provides whom are understandably reluctant to release packet captures. As a result, Modbus datasets that are publicly available are often narrowly scoped, synthetically simplistic, or lack operational diversity, particularly with respect to function codes, timing behaviors, and request-response semantics.

Supervisory Control and Data Acquisition (SCADA) networks present unique challenges for both security research and traffic modeling. Unlike enterprise networks, SCADA systems operate under strict protocol semantics, deterministic request-response patterns, and timing constraints that reflect physical processes. Protocols such as Modbus are foundational to these systems. Recent work has explored generative approaches for network traffic synthesis, including diffusion-based models that operate on packet-level representations. While these techniques show promise in enforcing low-level protocol structure, applying them to SCADA traffic introduces additional complexity. In SCADA environments, payload semantics, transaction consistency, and response behavior are just as critical as header correctness. As a result, synthetic traffic must be evaluated for both structural validity and behavioral realism.

In this work, we investigate the generation of realistic Modbus-based SCADA traffic through a combination of traffic fingerprinting, diffusion-based generation, and automated validation. We analyze both synthetic and real-world SCADA datasets, including Modbus used as a transport layer for SEL ASCII command streams, to characterize protocol behavior and timing patterns. We then evaluate NetDiffusion, a popular synthetic packet generator, culminating in the development of our custom diffusion pipeline designed to preserve request-response semantics while enabling dataset expansion.

To assess the quality of the generated traffic, we introduce a multi-stage validation pipeline. First, protocol-level checks verify Modbus transaction correctness and behavioral consistency against a virtual server. Second, a statistical similarity is evaluated using dimensionality reduction and supervised classification to measure the distinguishability of synthetic traffic from real SCADA traffic. Together, these methods provide a practical framework for generating and evaluating synthetic SCADA datasets suitable for security research, testing, and eval-

uation.

## 2 Related Work

Fingerprinting and generating network traffic to support network attack research across industrial control systems is not a new concept. The Computer Science Laboratory at SRI International investigated a signature-based approach for Modbus TCP in RTU mode [4]. Erez & Wool [5] contributes a fingerprint-adjacent classification model to detect Modbus register classes. Finally, Al-Dalky et al. [1] creates a Modbus traffic generator that generates malicious packets in order to test the effectiveness of intrusion detection systems.

nPrint [6] is a tool that creates fingerprints for network traffic. This tool works by extracting information from IPv4, IPv6, TCP, UDP, ICMP, and payload data. This works great for regular network traffic, however, one of its limitations is that it doesn't support SCADA protocols. That is the gap we would like to address. We want to create a fingerprinting tool that fingerprints SCADA protocols.

Our work focuses on creating fingerprints for SCADA Modbus traffic and generating benign Modbus TCP traffic to lessen the need for industry cooperation. We believe our work extends past research, and we plan to extend these models to include other SCADA protocols in the future.

## 3 Architecture/Methodology

### 3.1 Datasets

This project utilizes two significantly different Modbus datasets, one synthetic open-source and another proprietary from a real ICS. Our open-source dataset[3] contains Modbus traffic from a simulated substation network. We chose this specifically because we were struggling to find Modbus data that contained a variety of payloads (specifically function codes) to build and test our diffusion models. This dataset has an excellent spread of communications that makes packet generation much more interesting and universal to the Modbus protocol. This dataset contains over 220,000 Modbus packets that hold payloads for all reading operations (Read Discrete Inputs, Read Coils, etc.). Unfortunately, while this is the best spread of function codes we could find, even this dataset is missing write operations (e.g., Write Single Register). We anticipate this is due to Modbus being primarily used to monitor system state, rather than change it.

The second Modbus dataset includes traffic from a real-world industrial control system. It contains packets of many different protocols from GOOSE to ICMP, with about a fifth of the roughly 500,000 packets containing TCP headers. Much of this TCP traffic includes SEL commands (see Section 3.3) that use Modbus in ASCII mode as a transport layer. The two Modbus datasets complement each other nicely, since they show use cases for Modbus in both ASCII and RTU mode [11] and come from both real and synthetic systems.

### 3.2 Traffic Analysis

**Network Traffic Summary (Total endpoints: 7)**

| Host IP | Dst Hosts | Stream Count | Avg Packets/Stream |
|---|---|---|---|
| 185.175.0.5 | 1 | 14105 | 10 |
| 185.175.0.3 | 3 | 3 | 220619 |
| 185.175.0.4 | 2 | 14101 | 13 |
| 185.175.0.8 | 1 | 14104 | 13 |
| 185.175.0.1 | 1 | 1 | 31 |
| 224.0.0.251 | 0 | 0 | 0 |
| 185.175.0.6 | 0 | 0 | 0 |

Figure 1: Network analysis of the Modbus data The hosts with a stream count of 0 don't send out any packets, they only receive packets.

Our first goal was to analyze SCADA network traffic to characterize the network. We would like to determine the network topology, network endpoints, role of the endpoints, network events, and packet characteristics. The end goal would be to create fingerprints for all possible events in the datasets.

To determine the network endpoints, we look for unique IP sources and destinations in all the packets. Next, for each endpoint we figure out how many streams of communication they're a part of. We determine a stream by checking the four tuple (src ip, src port, dst ip, dst port), associated with each packet. To determine the role of each host we would need to do more analysis on the packets to figure out if they are a data or control packet. Depending on what type of packet each host sends or receives we could assume what their role is in the network. We did not get this far during this project.

Finally, we did analysis on the packets for each stream. We calculated the average packet size, inter arrival times, and average packets sent per stream. With this information we could make small assumptions about the roles of certain endpoints are. Figure 1 shows that the network contains 7 endpoints, what is interesting is that 2 of the hosts only receive packets, but never send packets, it can be assumed that they are only collecting data from the other hosts. We can also see some of the endpoints have a large amount of streams, however, these are short communications because they have low average packet count per stream. These sorts of endpoints might be sensors that are constantly creating new streams to send data frequently. Finally, we have one endpoint with

a low amount of streams, but these streams are persistent connections, due to the amount of packets per stream.

In conclusion, from the data we analyzed we could make some conclusions about the roles of certain endpoints in the networks. However, in order to be certain, we still need to do more packet analysis to see what kind of packets each endpoint is sending and receiving.

## 3.3 Fingerprinting Modbus over SEL

Schweitzer Engineering Laboratories (SEL) designs and manufactures products for industrial control systems, such as microprocessor relays for metering, fault locating, event recoding, and more. To communicate with these relays, they developed an open protocol that uses ASCII plain-language to issue commands. A user may send a sequence of commands such as "ACCESS", "OTTER", "METER", and "QUIT", to log in with the password "OTTER", display meter information, and exit. The simplistic nature of this open command protocol allows for integration with many different messaging protocols.

We extend our traffic analysis to a specific use case for the Modbus protocol: as a transport layer for SEL ASCII commands. We have obtained a proprietary dataset from a real industrial control system that utilizes Modbus to communicate with relays. We use this dataset to create fingerprints of unique ASCII streams to assist with our generated packet validation in the future.

Each Modbus packet carrying SEL data typically holds a single ASCII character in its payload. We have observed a few instances of multiple characters per packet, but this is not common. For example, as a user types "ACCESS" into the control terminal, the packets would contain "A", "C", "C", "E", "S", "S".

### 3.3.1 Fingerprinting script

We heavily utilize the Scapy[2] Python library to read and interpret packets. Our analysis script starts by reading the pcap file path passed as a command-line argument. It then begins extracting the TCP streams by using a dictionary with keys of 4-tuple (src_ip, src_port, dst_ip, dst_port), and values of lists of packets. Once we have all TCP streams, we filter for only streams that hold SEL ASCII commands in the payloads. To do this, we concatenate all packet data and attempt to interpret it as ASCII characters. If this ASCII sequence does not contain any SEL commands[10], filter it out. This function returns a new dictionary that uses the SEL ASCII sequence as the key, and a list of packet streams (list of lists) as the value. This groups all streams with the same character sequence together, allowing us to fingerprint each unique stream of characters. Finally, we analyze each ASCII sequence and consider each TCP stream individually. For each stream within a unique sequence, we record the following properties:

- Average, maximum, and minimum inter-arrival times
- Variance of inter-arrival times
- Unique packet sizes
- Stream durations
- TCP flag sequences
- Number of packets

By recording these results per stream, we are able to to analyze the differences between different streams that send the exact same commands.

```
== UNIQUE STREAM ==
QUIT\r\nACCESS\r\nOTTER\r\nSHOWSET\r\nQUIT\r\n
Number of streams with this sequence: 3
Number of packets in each stream: [74, 77, 74]
Duration of each stream (seconds): [14.29, 14.067, 13.951]
Unique packet sizes: {72, 66}
Flag distributions: [['S', 'A', 'PA', 'PA', 'PA', 'PA', 'PA', 'PA', 'A', 'PA', 'PA',
Average packet inter-arrival times (seconds) per stream: [0.1958, 0.1851, 0.1911]
Variance of packet inter-arrival times (seconds) per stream: [0.1938, 0.1808, 0.1886]
Maximum packet inter-arrival times (seconds) per stream: [3.8041, 3.7417, 3.758]
Minimum packet inter-arrival times (seconds) per stream: [0.000999, 2e-06, 0.001449]
```

Figure 2: Example SEL ASCII sequence fingerprint from analysis output.

### 3.3.2 SEL Analysis

Figure 2 highlights a snippet from the output of our analysis script. This example shows a sequence of commands "QUIT", "ACCESS", "OTTER", "SHOWSET", "QUIT", all separated by the carriage return (`\r`) and a newline (`\n`). This specific sequence ensures a fresh session ("QUIT"), logs in ("ACCESS") with a password ("OTTER"), displays active relay group settings ("SHOWSET"), then logs out ("QUIT").

For this proprietary dataset, we can see this particular sequence of commands occurred in three separate TCP streams. Each list in the analysis output shows the specific property for the corresponding stream in order. For example, the first stream with this ASCII sequence had 74 packets, a duration of 14.29 seconds, etc. Analyzing the example in Figure 2, we see that one of three streams had 77 packets, while the other two had 74 packets. As mentioned previously, occasionally packets will contain multiple ASCII characters. This small discrepancy in size between streams is normal. Looking at all inter-arrival time metrics and stream durations, we can see that these three streams had incredibly similar timings. This is excellent for our fingerprinting, and very likely indicates that this sequence is frequently executed by some script, not typed by a human. Another key observation

of this example is the sequence of PA (PSH/ACK) flags. After the TCP handshake, all ASCII payloads sent include PSH/ACK flags. This is apparent in all sequences, and is a defining property of SEL Modbus traffic.

We fingerprinted 13 unique ASCII streams from our dataset. In general, we saw low variance in timings, numbers of packets, and TCP flags. A few of our recorded sequences do show a relatively high degree of variability between timings especially. For example, for the single-command ASCII sequence "QUIT\r\n", we observed stream durations ranging from 0.6 seconds to 50.6 seconds. The most likely answer for this variance is that some streams are run through scripts, while others are manual entry.

The SEL command "ACCESS" is used to prompt the user to enter a password for login. We observed a strange pattern across multiple unique ASCII sequences where this login command is interpreted as "ACCES", missing the final "S". This only occurs when "ACCES" is followed by a carriage return and is missing a newline. To be clear, we observed either "ACCES\r" or "ACCESS\r\n". This must be an error in the packet reconstruction, as the sequences themselves indicate that the session was successful.

These fingerprints will be used in the future to ensure the similarity between synthetic and real-world traffic.

## 3.4 NetDiffusion

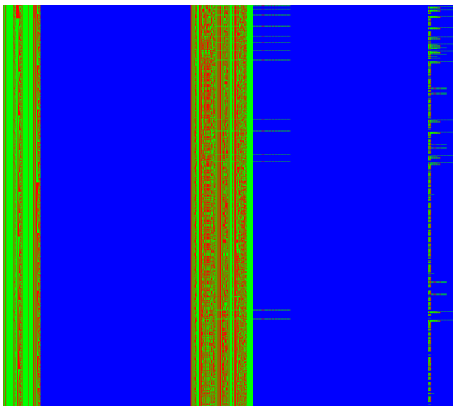

Figure 3: SEL Modbus nPrint converted to PNG.

Our second goal is to generate synthetic network traces that strictly adhere to the protocol specifications of the source protocol. This goal is an excellent application of NetDiffusion[9], which takes a source pcap file and generates synthetic, high-fidelity traffic that matches protocol semantics.

To begin, our source pcap is converted to a unified bit representation using nPrint[7]. This .nprint file displays each packet broken into bits representing each possible field, where 1 and 0 means the bit is set or unset, and -1 means the bit is missing. For example, if a given packet does not have any UDP headers, those columns will all be set to -1. In this file, each column represents a field and each row a packet.

With the traces in their bit representation, this file is converted to a PNG image. As shown in Figure 3, the blue pixels represent missing fields, where green and red represent set and unset bits respectively. From left to right, this figure shows data in IPv4, TCP, and payload columns, where the payload can be subtly observed on the far right. This image data is then fed into Low-Rank Adaption (LoRA) [8], which fine tunes Stable Diffusion 3.0 on the image data. LoRA is used due to the efficient training speed and model size, allowing for much lighter deployment and storage requirements.

After the model fine-tuning, NetDiffusion overlays a ControlNet[12] mask to the image that encourages the model to only generate pixels within the boundaries of the mask, significantly improving (but not guaranteeing) protocol compliance. Finally, critical fields that should not be generated and require uniformity across traces are systematically reconstructed (IP addresses, ports, etc.). This data is then converted from a pixilated network traffic image back into its nPrint bit representation, then finally reconstructed back into a pcap file.

### 3.4.1 Synthetic difficulties

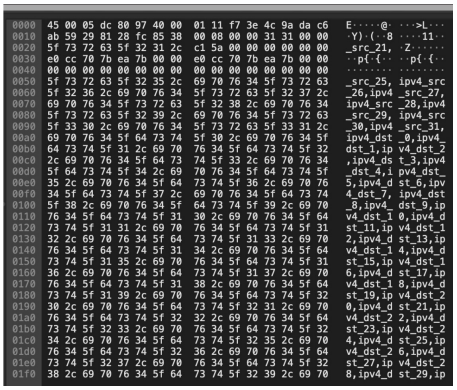

Figure 4: Malformed payload in NetDiffusion reconstructed pcap.

We ran our NetDiffusion tests on a high-memory Google CoLab workspace with a single L4 GPU. With this setup, we set out with the goal of creating synthetic network traffic that reconstructs our SEL Modbus and MQTTProvider datasets mentioned in Section 3.1.

It would be an understatement to say that we ran into a few issues with this setup. Mostly, we found a significant lack in documentation on how to actually use the

NetDiffusion framework. The paper goes into excellent detail on how the authors created the framework, what it can be used for, and their impressive results, but nowhere do they explain how a user can adapt their code to ever-so-slightly different applications. Their Python notebook documents itself well enough to follow through and understand each step, but any adjustments left us lost. We found this lack of documentation to be the root of all of our issues with the framework.

NetDiffusion does not consider payload columns by default, primarily since payloads are usually encrypted[9]. This project requires payload generation to some degree, so this was a significant challenge. All formatting nprint files, control images, and post-processing steps assume the data does not contain a payload. To remedy this, we attempted to patch all formatting, create our own control image, and adjust all of the dimensions the NetDiffusion code expects. This caused issues ranging from segmentation faults (while running Python in a cloud workspace) to completely uninterpretable pcap reconstructions. These patches caused multiple possible points of failure which were incredibly challenging to debug with minimal stack traces.

The closest we got to generating reasonable traces with NetDiffusion contained completely malformed payload data, to the point where many packets were over 1500 bytes, containing the ASCII text of the actual nprint header column names (pictured in Figure 4). Though this outcome is slightly humorous, we were left with little idea of where to begin debugging.

We may have concluded our NetDiffusion work with malformed pcap reconstructions and many insights into the improvements required to make future work much easier. Section 5.3 covers the future work required to better utilize the NetDiffusion framework.

## 3.5 Custom Diffusion Model Pipeline

### 3.5.1 Subtask

This part of the project focuses on generating realistic Modbus/TCP network traffic using a diffusion model trained on real PCAP data. The goal is to learn the statistical patterns of Modbus request–response exchanges and produce synthetic traffic that preserves protocol-level structure while allowing dataset expansion and privacy safe analysis.

Modbus traffic poses unique challenges because packets must follow strict formatting (MBAP header + PDU fields), requests must match responses, and timing behavior must be consistent. My subtask involved building a complete pipeline from raw PCAP parsing to training a diffusion model and generating synthetic PCAP output.

### 3.5.2 Dataset Preparation and PCAP Parsing

Raw PCAP files were converted into structured Modbus transaction records. Only TCP packets on port 502 were considered, and request packets were matched with their corresponding responses using the Modbus transaction identifier. For each transaction, the following features were extracted:

- request–response latency
- register address
- quantity of registers
- value field
- request payload length
- response payload length
- Modbus exception indicator
- response byte count
- Normalized response data length
- Normalized time from response to the next request
- One-hot encoded Modbus function code

These fields capture the operational semantics of Modbus traffic, allowing the diffusion model to learn realistic behavior. All features were normalized to the range $[0,1]$ via min–max scaling, and the original ranges were saved for inverse transformation during sampling.

### 3.5.3 Diffusion Model Architecture

The denoising network used in this work is a fully connected multilayer perceptron (MLP) trained to predict the Gaussian noise added to each feature vector during the forward diffusion process. The model is conditioned on the diffusion timestep through a sinusoidal time embedding. **Time Embedding** Each diffusion timestep $t$ is mapped to a high-dimensional representation using sinusoidal positional encodings:

$$\gamma(t) = \left[\sin(2^0\pi t), \cos(2^0\pi t), \ldots, \sin(2^k\pi t), \cos(2^k\pi t)\right].$$

This embedding produces a fixed-length vector (e.g., 64 dimensions) which is concatenated with the noisy feature vector $x_t$. The total model input dimension is therefore

$$d_{\text{input}} = d_{\text{features}} + d_{\text{time_embed}}.$$

**MLP Architecture** The reverse diffusion model consists of the following layers:

- Input layer of dimension $d_{\text{input}}$,

- Hidden Layer 1: 256 units with ReLU activation,
- Hidden Layer 2: 512 units with ReLU activation,
- Hidden Layer 3: 512 units with ReLU activation,
- Hidden Layer 4: 256 units with ReLU activation,
- Output Layer: $d_{\text{features}}$ units with Tanh activation.

ReLU activations are used to introduce nonlinearity in the hidden layers, while the final Tanh activation constrains the output to the range $[-1,1]$, consistent with the normalized feature space used during training.

**Model Role in Reverse Diffusion** The network outputs $\hat{\varepsilon}_\theta(x_t,t)$, an estimate of the noise added at timestep $t$. This estimate is used in the DDPM reverse update equation:

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(x_t - (1-\alpha_t)\hat{\varepsilon}_\theta(x_t,t)\right) + \sigma_t z,$$

where $z \sim \mathcal{N}(0,I)$ for $t > 1$. Repeated application of this update reconstructs a clean feature vector representing a synthetic Modbus transaction.

#### 3.5.4 Training

Training proceeds by uniformly sampling a timestep $t$ for each batch, adding Gaussian noise to the clean feature vectors according to the forward diffusion process, and training the neural network to predict this noise. The model was trained for 50 epochs with a batch size of 512 and a learning rate of $1 \times 10^{-3}$ using the Adam optimizer. The diffusion process consisted of 1000 timesteps with a linear variance schedule ranging from $\beta_{\text{start}} = 1 \times 10^{-4}$ to $\beta_{\text{end}} = 2 \times 10^{-2}$.

#### 3.5.5 Synthetic Sample Generation

Synthetic Modbus transactions are produced by initializing with pure Gaussian noise and iteratively applying the reverse diffusion process from $t = T$ to $t = 1$. The resulting feature vector is clipped to the range $[0,1]$ and then denormalized using the stored real data feature ranges. Continuous outputs corresponding to discrete Modbus protocol fields (e.g., function codes, addresses, quantities) are discretized to yield a valid reconstructed transaction. These synthetic records can subsequently be assembled into full PCAP files.

## 4 Results/Evaluation

### 4.1 Feature Distribution Comparison

The feature-distribution results in Table 1 and Table 2 show that the diffusion model closely reproduces the statistical properties of most normalized Modbus features. Payload lengths, byte counts, data lengths, and quantity fields exhibit similar means and variances between real and synthetic samples, indicating that the model successfully captures the core structure of Modbus transactions. Timing-related features (e.g., `dt_req_resp` and `dt_next_req`) display slightly larger deviations, which is expected because timing is influenced by external network factors not modeled directly by the diffusion process.

Table 1: Normalized feature distribution comparison (mean and standard deviation).

| Feature | Set | Mean | Std |
|---|---|---|---|
| dt_req_resp | real | -0.9997 | 0.0151 |
| | synth | -0.9666 | 0.0318 |
| addr_norm | real | -0.2856 | 0.4469 |
| | synth | -0.3494 | 0.1345 |
| qty_norm | real | 1.0000 | 0.0000 |
| | synth | 0.9829 | 0.0273 |
| val_norm | real | -0.9980 | 0.0634 |
| | synth | -0.9727 | 0.0351 |
| req_payload_len | real | -0.9077 | 0.0000 |
| | synth | -0.8851 | 0.0239 |
| resp_payload_len | real | -0.9200 | 0.0038 |
| | synth | -0.8977 | 0.0304 |
| bytecount_norm | real | -0.9889 | 0.0039 |
| | synth | -0.9649 | 0.0358 |
| data_len_norm | real | -0.9889 | 0.0039 |
| | synth | -0.9789 | 0.0239 |
| dt_next_req_norm | real | -0.9759 | 0.1897 |
| | synth | -0.9393 | 0.0389 |

### 4.2 Function Code Distribution

The function code distribution demonstrates strong operational fidelity: synthetic data preserves the dominance of FC 1–4 observed in real Modbus traffic. Low-frequency codes (FC 5 and FC 6) do not appear in the synthetic set, reflecting their extremely limited representation in the training data. Overall, the results indicate that the model captures the functional patterns well.

### 4.3 Validation Pipeline

After generating a synthetic pcap file, the traffic is passed through an automated validation pipeline designed to verify the protocol correctness and the statistical fidelity with respect to real Modbus traffic. Validation is performed in two stages:

1. Protocol-level verification

Table 2: Normalized feature distribution comparison (min and max values).

| Feature | Set | Min | Max |
|---|---|---|---|
| dt_req_resp | real | -1.0000 | 1.0000 |
| | synth | -0.9946 | -0.5726 |
| addr_norm | real | -0.7143 | 1.0000 |
| | synth | -0.5937 | 0.0773 |
| qty_norm | real | 1.0000 | 1.0000 |
| | synth | 0.6225 | 1.0000 |
| val_norm | real | -1.0000 | 1.0000 |
| | synth | -1.0000 | -0.6731 |
| req_payload_len | real | -0.9077 | -0.9077 |
| | synth | -0.9210 | -0.6666 |
| resp_payload_len | real | -0.9231 | -0.9077 |
| | synth | -0.9321 | -0.5769 |
| bytecount_norm | real | -1.0000 | -0.9841 |
| | synth | -1.0000 | -0.6256 |
| data_len_norm | real | -1.0000 | -0.9841 |
| | synth | -1.0000 | -0.7280 |
| dt_next_req_norm | real | -1.0000 | 50.2356 |
| | synth | -0.9999 | -0.6237 |

Table 3: Function code distribution comparison.

| Func. Code | Real Count (%) | Synthetic Count (%) |
|---|---|---|
| FC 1 | 43984 (0.399) | 2049 (0.410) |
| FC 2 | 21991 (0.199) | 1431 (0.286) |
| FC 3 | 22021 (0.200) | 829 (0.166) |
| FC 4 | 21995 (0.199) | 691 (0.138) |
| FC 5 | 111 (0.001) | 0 (0.000) |
| FC 6 | 216 (0.002) | 0 (0.000) |

2. Statistical indistinguishability testing

#### 4.3.1 Protocol-Level Verification

Each generated packet is first validated against fundamental Modbus/TCP constraints. Packets are converted into raw byte representations and checked for structural consistency. Specifically, the pipeline verifies:

- Transaction ID consistency between request and response pairs.
- Function code agreement, including detection of Modbus exception responses.
- Payload presence and application-layer integrity in response packets.

To verify behavioral correctness, each generated request packet is transmitted to a virtual Modbus server using Scapy. The server's response is then compared against the corresponding generated response to identify any protocol violations. This step ensures that the synthetic packets are not just structurally valid, but also behaviorally consistent with real Modbus interactions.

#### 4.3.2 Statistical Validation via PCA and SVM

Once protocol correctness has been verified, synthetic packets are evaluated for their statistical similarity to real Modbus traffic. The goal of this stage is to determine whether a machine-learning classifier can reliably differentiate synthetic packets from real ones.

Both real and synthetic packets are converted into pandas DataFrames using the same feature extraction pipeline. Real and synthetic samples are balanced, labeled, merged, shuffled, and split into training and testing datasets using an 80/20 train-test split.

Before classification, all features are normalized, using a standard z-score scaling. To reduce the dimensionality and remove redundant correlations, Principle Component Analysis (PCA) is applied, retaining 95% of the variance. This reduces the original 13 features down to 5 principle components while maintaining most of the statistical structure of the data.

A linear Support Vector Machine (SVM) is then trained on the PCA-reduced training set and evaluated on the test set. If the classifier achieves near-random performance ( 50% accuracy), this implies that the generated packets are statistically indistinguishable from real traffic. Conversely, a high classification accuracy indicates that the synthetic traffic contains identifiable artifacts.

### 4.4 Classification Results

```
Original number of features: 13
Number of features after PCA: 5
Accuracy: 0.7846
Confusion Matrix:
 [[968   5]
 [372 405]]
Classification Report:
              precision    recall  f1-score   support

           0       0.72      0.99      0.84       973
           1       0.99      0.52      0.68       777

    accuracy                           0.78      1750
   macro avg       0.86      0.76      0.76      1750
weighted avg       0.84      0.78      0.77      1750
```

Figure 5: PCA Results, Confusion Matrix, and Classification Report.

The trained SVM achieved an overall classification accuracy of 78.46%, indicating that the model was able to distinguish between real and synthetic packets, but not with high confidence. The resulting confusion matrix and classification metrics are shown in Figure 5.

Table 4: Top SVM feature weights distinguishing real vs synthetic samples.

| Feature | Weight |
|---|---|
| response_time | 5.6311 |
| byte_cnt | 0.0497 |
| func_code | 0.0375 |
| bitnum | 0.0211 |
| reference_num | 0.0094 |

A closer inspection of the confusion matrix reveals an encouraging outcome, approximately 47.9% of the synthetic packets were misclassified as real, demonstrating that a substantial subset of the generated traffic is indistinguishable from authentic Modbus communication. This result is significant, as the classifier fails to reliably detect a large fraction of synthetic traffic despite operating in a reduced feature space.

Additionally, as seen in Table 4, `response_time` receives a weight much larger than all other features, indicating that it is the primary factor enabling discrimination between real and synthetic samples. The remaining features have very small weights, consistent with the strong overlap between their real and synthetic distributions.

While perfect indistinguishability was not achieved, these results demonstrate that the generator is already capable of generating traffic that meaningfully overlaps with regions of the real Modbus feature space. This provides a solid empirical foundation for future refinement, which we'll discuss further in the next section.

# 5 Future Work

## 5.1 Model Improvements

The primary limitation in the current results is that response_time remains the most challenging feature to reproduce accurately. Unlike other Modbus fields, which the diffusion model learns well, the synthetic timing values show a noticeable gap compared to real traffic. A promising direction for future work is to treat response_time as its own independently modeled component. Separating timing from the other semantic fields may allow a specialized model to learn timing patterns more effectively and improve overall realism.

## 5.2 Extending Network Analysis to Real-World SCADA datasets

Our current work in network analysis and fingerprinting only works with the Modbus and SEL protocols. However, We would like to extend this work to the Real-World SCADA datasets we have.

The challenges of Real-World SCADA traffic, is that it contains multiple protocols that are hard to work with because of variation in implementations and the protocol specifications not being easily available to the public. This would not only require us to make our tools work for multiple protocols rather than just focusing on one or two, but we would also have to do some manual work to understand the protocols that we don't have access to their specifications.

## 5.3 Other Work

Through our struggles as NetDiffusion users, we gained some valuable insights into how this framework could be improved for future work. To continue with this project, a far deeper understanding of NetDiffusion is required. The primary Python notebook contains cells with documentation explaining what NetDiffusion does at each step in the pipeline. While this does help build some background knowledge, we could not find resources for NetDiffusion from a user prospective. Documentation that outlines each step of the process with insights on particular adjustments required to use the pipeline on different types of network traces would be an excellent start. A significant area of trouble was understanding and creating a ControlNet image to replace the default one included in the NetDiffusion repository. We believe a straightforward function to generate a new control image should be a key part of the pipeline. In general, we believe the code is mostly tailored to examples, rather than real use. With contributions from the user prospective, we are certain NetDiffusion will be invaluable for synthetic traffic generation.

# 6 Impact and Applicability

As mentioned previously, it can be difficult for researchers to obtain datasets that are both high quality and sufficiently diverse. This tool lowers the barrier to entry for ICS research and experimentation, enabling researchers to focus on protocol fingerprinting and anomaly detection instead of finding useful datasets. By ensuring the semantic validity of the packets, we ensure that the generated traffic is suitable not just for machine-learning experiments, but also for protocol testing, simulator input, and defensive tool evaluation where behavioral realism is critical.

From a security perspective, the ability to produce large volumes of controllable synthetic traffic has practical implications for defensive testing and red-team/blue-team exercises. Synthetic Modbus traces can be use to stress-test intrusion detection systems, evaluate the

robustness of statistical classifiers, and explore failure modes in detection pipelines without risking disruptions to real operational systems. The fact that a non-negligible portion of the generated packets were indistinguishable from real traffic reinforces the realism and potential usability of this approach. Beyond security research, this generator is applicable to education and training as well. ICS concepts can be difficult to teach due to the lack of accessible datasets and safe experimentation environments, an open-source Modbus generator enables students and practitioners to explore industrial protocols hands on.

Finally, this work contributes to the broader goal of open, reproducible ICS research. By releasing this generator and validation pipeline, we provide a foundation that can be extended to additional protocols, refined with improved diffusion-based techniques, or adapted to organization-specific modeling needs. While synthetic data is not a replacement for real-world captures, tools like this help bridge the long-standing gap between theoretical research and the realities of critical infrastructure needs.

## 7 Acknowledgments

Thank you to Dr. Ram Sundara Raman for giving great feedback throughout this project and for helping us when we frequently got stuck. This was challenging work that would not have been possible otherwise.

While we struggled with NetDiffusion as a tool, the insights provided by the researchers who developed it proved to be invaluable. We relied on both the tool itself and its associated paper when designing our diffusion-based packet generator.

## 8 Availability

Our work is available at

`https://github.com/char26/scada-generator`

## References

[1] AL-DALKY, R., ABDULJALEEL, O., SALAH, K., OTROK, H., AND AL-QUTAYRI, M. A modbus traffic generator for evaluating the security of scada systems. In *2014 9th International Symposium on Communication Systems, Networks Digital Sign (CSNDSP)* (2014), pp. 809–814.

[2] BIONDI, P., AND THE SCAPY COMMUNITY. Scapy - The Interactive Packet Manipulation Tool, 2025. Project started in 2008.

[3] BOAKYE-BOATENG, K., GHORBANI, A. A., AND LASHKARI, A. H. Securing Substations with Trust, Risk Posture and Multi-Agent Systems: A Comprehensive Approach. In *20th International Conference on Privacy, Security and Trust (PST)* (Copenhagen, Denmark, aug 2023).

[4] CHEUNG, S., DUTERTRE, B., FONG, M., LINDQVIST, U., SKINNER, K., AND VALDES, A. Using model-based intrusion detection for scada networks. In *Proceedings of the SCADA Security Scientific Symposium* (Miami Beach, Florida, Jan. 2007).

[5] EREZ, N., AND WOOL, A. Control variable classification, modeling and anomaly detection in modbus/tcp scada systems. *International Journal of Critical Infrastructure Protection 10* (2015), 59–70.

[6] HOLLAND, J., SCHMITT, P., FEAMSTER, N., AND MITTAL, P. New directions in automated traffic analysis. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security* (New York, NY, USA, 2021), CCS '21, Association for Computing Machinery, p. 3366–3383.

[7] HOLLAND, J., SCHMITT, P., FEAMSTER, N., AND MITTAL, P. New Directions in Automated Traffic Analysis. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS '21)* (Virtual Event, Republic of Korea, nov 2021), ACM, pp. 3366–3383.

[8] HU, E. J., SHEN, Y., WALLIS, P., ALLEN-ZHU, Z., LI, Y., WANG, S., AND CHEN, W. Lora: Low-rank adaptation of large language models. *CoRR abs/2106.09685* (2021).

[9] JIANG, X., LIU, S., GEMBER-JACOBSON, A., BHAGOJI, A. N., SCHMITT, P., BRONZINO, F., AND FEAMSTER, N. Netdiffusion: Network data augmentation through protocol-constrained traffic generation. *Proceedings of the ACM on Measurement and Analysis of Computing Systems 8*, 1 (2024), 1–32.

[10] SCHWEITZER, E. O., AND SCHEER, G. W. SEL Products and Features Make Integration Easy. Technical Paper/Application Note (Schweitzer Engineering Laboratories), 1999.

[11] WITTE SOFTWARE. Modbus Protocol Description, 2025.

[12] ZHANG, L., RAO, A., AND AGRAWALA, M. Adding Conditional Control to Text-to-Image Diffusion Models. *arXiv e-prints* (nov 2023).